CMPE 492

Auto-Curriculum: Does a Network Choose to See Blurry First?

Ömer Faruk Çelik

Mustafa Taha Söylemez

Advisor:

Suayb S. Arslan

TABLE OF CONTENTS

1. INTRODUCTION 1
1.1. Broad Impact 2
1.2. Ethical Considerations 2
2. PROJECT DEFINITION AND PLANNING 3
2.1. Project Definition 3
2.2. Project Planning 3
2.2.1. Project Time and Resource Estimation 3
2.2.2. Success Criteria 4
2.2.3. Risk Analysis 4
2.2.4. Team Work (if applicable) 5
3. RELATED WORK 6
4. METHODOLOGY 8
5. REQUIREMENTS SPECIFICATION 11
6. DESIGN 12
6.1. Information Structure 12
6.2. Information Flow 12
6.3. System Design 13
6.4. User Interface Design (if applicable) 13
7. IMPLEMENTATION AND TESTING 14
7.1. Implementation 14
7.2. Testing 14
7.3. Deployment 15
8. RESULTS 16
9. CONCLUSION 21
REFERENCES 22
APPENDIX A: FULL RESULT TABLES AND EXPERIMENT LOG 25

1. INTRODUCTION

A newborn sees the world blurry and with low contrast. Visual acuity and contrast sensitivity improve over the first months of life. One view says that this slow start is useful, not a defect: the visual system first learns coarse shape and only later fine texture, and this may make adult vision more robust to degraded input [1]. Several studies tested this idea by training convolutional networks on a hand-made blurry-to-sharp schedule and measuring robustness [1–3]. In all of them a researcher writes the schedule. Our project asks one step back: *if the network chooses the blur and contrast of its own training images, based only on how fast it learns, does it start blurry and faint like an infant? And does that make it more robust?*

We use a teacher from automated curriculum learning [4]: a non-stationary multi-armed bandit whose arms are (blur, contrast) levels. Before each training step the teacher picks an arm, the batch is degraded at that level, and the network takes one SGD step. The reward is learning progress on the training data only. Robustness, shape bias and test data never enter the reward; we measure them only after training. If robustness were in the reward, the teacher would optimise it directly, and we could not say that an order "emerged". We test two hypotheses. **H1 (an infant-like order emerges):** a learning-progress teacher picks blurry, low-contrast arms early in training and sharp arms late. **H2 (robustness follows):** the self-chosen schedule gives more robustness and more shape bias than controls, including a control that sees the same arms in a shuffled order.

Main findings (CIFAR-10, ResNet-18, three seeds). H1 is rejected: none of the seven teacher variants we tried started blurry. With the raw learning-progress reward the teacher starts near uniform and then moves to the most degraded arm. For H2, robustness and shape bias follow *exposure to blur*, not order. At 100 epochs, no ordered schedule beat a uniform random arm per batch on CIFAR-10-C. The direction of an order still matters: sharp-to-blurry is the worst order. Batch-level contrast training has almost no effect under BatchNorm; contrast is learned only with per-

image mixing or GroupNorm. Table 8.1 grades each claim. Chapters 2–3 give the plan and related work, Chapter 4 the method, Chapters 5–7 the experiment framework as software (requirements, design, implementation and testing), Chapter 8 the results and Chapter 9 the conclusion.

1.1. Broad Impact

The project is basic research; its main value is understanding. First, it tests a popular analogy between infant vision and curriculum learning with a design that separates order from exposure. The clear negative result for H1 tells the next study that a learning-progress teacher alone does not reproduce the developmental order, and why (the reward scale favours the hardest arm). Second, the results give practical guidance for robustness training: mixing blurred images throughout training was at least as good as any ordered schedule we tested, and batch-level contrast augmentation does not work with BatchNorm. Third, the code base is public and reproducible (fixed seeds, bit-exact resume, 52 tests), so other students can extend it. We do not expect a direct product. Robustness research can, in principle, help both good and bad uses of vision systems; our small CIFAR-10 study adds almost nothing to that risk.

1.2. Ethical Considerations

We use only public benchmark data: CIFAR-10 [5] and CIFAR-10-C [6]. There are no human subjects, no personal data and no data collection. The models are small image classifiers for ten object classes and are not deployed. The main cost is compute and the energy it uses: all runs together used a few hours of one cloud A100 GPU within a 300-compute-unit budget, and we kept runs short (30 and 100 epochs) and parallel to limit this. We report negative results and the limits of our evidence (three seeds, one dataset, one model) so that the claims are not stronger than the data. The repository is public and the advisor can inspect every step.

2. PROJECT DEFINITION AND PLANNING

2.1. Project Definition

The project builds an experiment framework and uses it to answer H1 and H2. The data are CIFAR-10 (50 000 training and 10 000 test images, 32×32 pixels) and the model is a CIFAR-style ResNet-18 [7]. The scope covers 16 (blur, contrast) arms; two teachers, Exp3.S [4, 8] with a learning-progress reward (prediction gain and variants) and ALP-GMM [9]; five controls (sharp only, random arm per batch, a hand-made blurry-to-sharp staircase, its reverse, and the teacher's own arm log shuffled in time); four measures (clean test accuracy, CIFAR-10-C split into "seen" and "unseen" corruptions, a frequency-based shape-bias proxy, and PGD adversarial accuracy); and two budgets, 30 epochs (short) and 100 epochs (full), with 3 seeds each. Chapter 4 defines each part. The deliverables are the public code repository[1], the result summaries, the experiment log, and this report.

2.2. Project Planning

2.2.1. Project Time and Resource Estimation

Table 2.1 gives the actual timeline: a reading phase, a proposal, and three short experiment phases. The plan named the experiment phases F0–F8 (set-up, pipeline, teacher, controls, pilot, short budget, full budget, robustness checks, analysis).

Resources. We used Colab Pro with a budget of 300 compute units and an A100 (40 GB) GPU. Before any measurement, the plan estimated about 225 of the 300 units, assuming about 6 s per epoch: F1–F2 development 15, F4 GPU comparison and pilot 10, F5 short budget (21 runs) 30, F6 full budget (21 runs) 90, F7 robustness checks 30, and a re-run reserve of 50. The real cost was much lower. One epoch takes 2.0 s in eager mode (`torch.compile` gives 1.7 s but costs 30 s to compile, so we did not use it). A full CIFAR-10-C evaluation takes 15 s. Because all data stays in GPU memory,

[1] `https://github.com/omarreis01/autofocus-curriculum`

Table 2.1. Project timeline (2026).

Date	**Work**
Early Oct. – Oct. 11	Reading: Graves 2017, EfficientTrain/++, Wu 2021, Hacohen 2019, Portelas 2019 and the advisor's CCN 2024 paper. Notes in `docs/okuma/`.
Oct. 13	Project proposal: H1/H2, seven conditions, CIFAR-10 + ResNet-18.
Nov. 3	Set-up (F0–F4): repository, Colab launcher, degradation, training, evaluation, teacher and controls (24 tests). First GPU run: sharp, 30 epochs, 93.2% clean. Pilot: 7 conditions, seed 0.
Nov. 4	BatchNorm problem found and fixed (recalibration before evaluation). F5: 7 conditions × 3 seeds × 30 epochs. F6 (100 epochs) started. Extended plan written.
Nov. 8	F6 finished. Extended code (52 tests). 13 new conditions × 3 seeds and probes on all 81 checkpoints.
Nov. 11	Full project report (Turkish).

6 runs share one A100 in parallel; one wave of 12 parallel runs ran out of memory (Table 2.2). Teacher runs need one extra forward pass per step for the reward.

2.2.2. Success Criteria

We set five criteria before the experiments; all were met. (i) *Pipeline is correct:* the sharp condition reaches about 93% clean accuracy after 30 epochs (known value for ResNet-18); result: 93.2%. (ii) *Teacher is sane:* with a network that does not learn, the teacher gives no reward and stays uniform; with one arm that pays more, it concentrates on that arm (both are unit tests). (iii) *Fair comparison:* every condition uses the same number of steps, the same learning-rate schedule and the same data order (tested). (iv) *Statistics:* every reported number is the mean ± standard deviation over 3 seeds, at a short and a full budget. (v) *A clear answer:* the report answers H1 and H2 either way, with evidence for each claim; a negative result counts as a result.

2.2.3. Risk Analysis

Table 2.2 lists the risks we planned for and the problems that happened.

Table 2.2. Risks, mitigations and what happened.

Risk	**Mitigation**	**What happened**
Gains come from blur exposure, not order [10, 11]	Shuffled control (same arms, random order), random and reverse controls	This was the key control; see Chapter 8.
Colab session ends during a run	Checkpoint every 5 epochs to Google Drive, bit-exact `--resume`	Happened in F6 (the PC was shut down near epoch 40); runs resumed.
A100 not available / budget overrun	Measure speed first (F4), keep runs short	A100 was available; measured cost far below estimate.
Slow dataset servers	Hugging Face mirrors, checked against the official files (CIFAR-10 archive by MD5; CIFAR-10-C label order against the official test labels)	Zenodo and Toronto were very slow; mirrors took ∼1.5 min.
Bandit does not move in 3k–10k steps	Larger learning rate η	With Graves' $\eta = 10^{-3}$ the policy did not move; we use $\eta = 0.01$.
Hidden bug in a mixed-arm pipeline	Pilot run before the main runs	BatchNorm mismatch found in the pilot (Chapter 4).
GPU out of memory	Fewer parallel runs	3 of 12 parallel runs (GroupNorm and per-image) died; re-run from checkpoint at 6 parallel (`colab/finish_ext.sh`). Same set-up, so results are not affected.
Small effects	3 seeds, two budgets, report std	Most differences are far above the std; the small ones are marked.

2.2.4. Team Work (if applicable)

The team has two members. For the reading phase, the work was split by topic: Ömer Faruk Çelik read the curriculum and bandit papers (Graves 2017, EfficientTrain/++, Wu 2021, Hacohen 2019, Portelas 2019) and wrote that part of the reading notes; the other topics of the reading notes were assigned to Mustafa Taha Söylemez. Both read the advisor's CCN 2024 paper. Both members wrote the code and ran the experiments together; the commits were made from one computer, so the repository history lists a single author. The advisor, Suayb S. Arslan, gave the research direction through his work on developmental noise and resolution schedules [3].

3. RELATED WORK

Automated curriculum with bandits. Graves et al. [4] let a non-stationary bandit, Exp3.S [8], choose which task to sample next. The reward is learning progress, for example the prediction gain (PG): the drop in loss on a batch after one update. They found that uniform sampling over tasks is a strong baseline, and warned that PG can favour arms with high gradient variance. We use this teacher directly, with image degradation levels as arms instead of tasks. Portelas et al. [9] proposed ALP-GMM, which fits a Gaussian mixture over a continuous task space and samples where the absolute learning progress is high; it is our second teacher.

Low-frequency-first training. EfficientTrain [12] and EfficientTrain++ [13] show the network low-frequency content first and the full image later. This speeds up ImageNet training by about 1.5–3× without accuracy loss. This is the closest prior work, but the schedule is found offline and then fixed, the goal is compute, and robustness is not measured. Our hand-made staircase is a version of this fixed schedule.

When do curricula help? Wu, Dyer and Neyshabur [10] trained thousands of models and found that curriculum, anti-curriculum and random order give the same result in standard training; small gains come from pacing, and curricula help mainly with short budgets or noisy labels. This is the main reason for our shuffled, random and reverse controls and for the two budgets. Hacohen and Weinshall [14] split a curriculum into a scoring function and a pacing function, and report small gains (1–2 points) from easy-to-hard order. In our design one bandit sets both order and pacing, so we keep the learning-rate schedule the same in every condition.

Developmental vision and blur. Vogelsang et al. [1] argue that initially poor acuity may be adaptive and support this with networks trained blurry-to-sharp. Katzhendler and Weinshall [11] replied that low-resolution images could simply be mixed in at all times; the benefit may come from exposure, not order. Jang and Tong [2] found that for objects (unlike faces) the robustness from blurry-to-clear training is not kept at the end of training; later [15] they showed that training with mixed blur

gives networks more shape sensitivity and more robustness to noise. Arslan, Jang and Sinha [3] showed that decreasing internal neural noise from high to low during training, and low-to-high resolution, improves robustness to degraded input. All of these schedules are set by hand (Table 3.1). Our question is whether the order appears when the network chooses. Contrast is our addition.

Robustness, shape and normalisation. CIFAR-10-C [6] applies 19 corruptions at 5 severities to the CIFAR-10 test set. Geirhos et al. [16] defined shape bias with style-transfer cue-conflict images; we use a simpler frequency proxy because that set does not fit 32×32 images. Wang et al. [17] showed that CNNs use high-frequency components that humans cannot see, which motivates our low-/high-pass probes. PGD [18] is the standard iterative adversarial attack. BatchNorm [19] normalises with batch statistics in training and with running averages at test time; GroupNorm [20] normalises within each sample and has no batch statistics. This difference turned out to be central for contrast (Chapter 4).

Table 3.1. Position of this project relative to the closest work.

Work	**Who sets the order**	**Axis**	**Measures robustness?**
Graves et al. [4]	Bandit, learning progress	Tasks (text, bAbI)	No
EfficientTrain/++ [12, 13]	Offline search, then fixed	Frequency, augmentation	No
Vogelsang; Jang & Tong [1, 2, 15]	Researcher	Blur	Yes
Arslan et al. [3]	Researcher	Internal noise, resolution	Yes (noisy input)
This project	**Bandit, learning progress**	**Blur and contrast**	**Yes (CIFAR-10-C, PGD, shape proxy)**

4. METHODOLOGY

Data, model and training. We train on CIFAR-10 with random crop (padding 4) and horizontal flip. Blur and contrast are applied on the GPU after augmentation. The model is ResNet-18 in its CIFAR form (3×3 stem, no max-pool) with BatchNorm; one group of experiments uses GroupNorm with 32 groups. We use SGD with Nesterov momentum 0.9, weight decay $5 \cdot 10^{-4}$, batch size 512 and bf16 mixed precision. The learning rate rises linearly to 0.2 over the first 15% of steps and then follows a cosine to zero; this schedule is the same in every condition. The short budget is 30 epochs (2 910 steps) and the full budget is 100 epochs (9 700 steps), with 97 steps per epoch. Every condition runs with seeds 0, 1 and 2.

Arms: blur and contrast. An arm is a pair (σ, c) from a grid of 4 blur levels ($\sigma \in \{0, 0.75, 1.5, 3\}$ pixels) $\times$ 4 contrast levels ($c \in \{1, 0.6, 0.35, 0.2\}$) = 16 arms; the arm $(0, 1)$ is the sharp image. Blur is a separable Gaussian filter with standard deviation σ pixels and reflect padding. Contrast then moves every pixel toward the image mean m of its channel, $x' = m + c\,(x - m)$. In all conditions except one (`random_image`), every image in a batch gets the same arm.

Teacher: Exp3.S with learning-progress rewards. At step t the teacher keeps weights w_i for the $K = 16$ arms and samples arm a_t from

$$\pi_t(i) = (1 - \epsilon)\,\frac{e^{w_i}}{\sum_j e^{w_j}} + \frac{\epsilon}{K}. \tag{4.1}$$

The batch is degraded with arm a_t, the network takes one SGD step, and the teacher measures a raw reward $\tilde{r}_t$. **PG** (prediction gain) is the loss on the same batch before the update minus the loss after it (one extra forward pass). **SPG** is the same difference on a fresh batch from the same arm; it removes the bias of measuring on the batch just trained on. **RPG** (relative PG) is PG divided by the loss before the update; it removes the scale of the loss. The raw reward is mapped to $[-1, 1]$ with the 20th and 80th percentiles q^{lo}, q^{hi} of the last 1000 raw rewards [4]:

$$r_t = \mathrm{clip}\left(2\,\frac{\tilde{r}_t - q^{lo}}{q^{hi} - q^{lo}} - 1,\ -1,\ 1\right). \tag{4.2}$$

The Exp3.S update with importance weighting is $w_{a_t} \leftarrow w_{a_t} + \eta\, r_t/\pi_t(a_t)$, followed by weight sharing with rate $\alpha_t = 1/t$ so that the teacher can track a changing best arm. We use $\eta = 0.01$, $\epsilon = 0.05$. Graves et al. use $\eta = 10^{-3}$ for about 10^5 steps; with our 3k–10k steps it did not move the policy (Table 2.2). The forward passes that measure the reward do not change the BatchNorm running statistics, so a teacher run differs from a fixed-arm run only in the choice of arms.

ALP-GMM. The second teacher samples a continuous point in the (σ, c) box (scaled to $[0,1]^2$), with 20% random samples and otherwise from a Gaussian mixture fitted every 250 steps on the absolute learning progress. Each sample is rounded to the nearest of the 16 arms for training.

Conditions. Table 4.1 lists the 20 conditions: the seven main ones ran at both budgets, the 13 extended ones at 30 epochs only.

Measures. Clean is CIFAR-10 test accuracy. **CIFAR-10-C** [6] has 19 corruptions × 5 severities. *Seen* is the mean over the 6 corruptions close to our training degradations (Gaussian, defocus, glass, motion and zoom blur, contrast); *unseen* is the mean over the other 13 (noise, weather, digital). We also report the mean of the 4 noise corruptions and the contrast corruption alone. **Shape bias (CIFAR proxy):** a hybrid image is the low-frequency part (Gaussian $\sigma = 2$) of an image of class A plus the high-frequency part of an image of class B; shape bias is the share of A answers among A or B answers, over about 10 000 pairs. This is not Geirhos' style-transfer test [16]; it measures reliance on low versus high frequencies. We also report accuracy on high-pass images ($\sigma = 1$). **PGD-10** [18] uses L_∞, $\epsilon = 1/255$, step $\epsilon/4$, random start, on 2 000 test images. We also measured FGSM but do not use it, because it looks gradient-masked: at 100 epochs, `sharp` scores higher under FGSM at 2/255 than under PGD at 1/255 (53.6 vs 44.9).

BatchNorm recalibration. In the pilot, `sharp` was normal, but every condition that mixes arms over time scored only 13–60% clean accuracy. The cause is BatchNorm. Each batch comes from one arm. In training, BatchNorm uses the statistics of the current batch, so this is fine. At test time it uses running averages, which are a mix of degraded batches and do not fit clean images. We tested this: after recomputing the running statistics on 100 batches of clean *training* images, `random` rose from 60 to

Table 4.1. Conditions and the question each one answers.

Condition	**What it does**	**What it tests**
Main seven (30 and 100 epochs)		
`sharp`	Always $(0, 1)$.	Lower bound: no blur seen.
`random`	Uniform random arm per batch.	Exposure without order.
`staircase`	Diagonal $(3, .2) \to (1.5, .35) \to (.75, .6) \to (0, 1)$ in 4 equal stages over the first 50% of steps, then sharp.	Hand-made blurry-to-sharp (what H1 expects).
`reverse`	Staircase reversed in time; same arm counts.	Does the direction matter?
`teacher_pg`	Exp3.S + PG.	Main method.
`teacher_spg`	Exp3.S + SPG.	Same-batch bias of PG.
`shuffled`	Arm log of `teacher_pg` (same seed), shuffled in time.	Same exposure, no order.
A: teacher variants (30 epochs)		
`teacher_rpg`	Exp3.S + RPG.	Is "hard first" caused by the PG scale?
`teacher_pg_eta0.003/0.03`	$\eta \times 0.3$ and $\times 3$.	Sensitivity to η.
`teacher_pg_noiw`	No importance weighting.	Update detail.
`teacher_alp`	ALP-GMM.	Does another teacher choose the same?
B: training regime (30 epochs)		
`random_image`	Random arm per image (mixed batch).	Per batch vs per image.
`sharp/random/teacher_pg_gn`	GroupNorm(32) instead of BatchNorm.	Do results depend on BatchNorm?
`random_blur`	Blur axis only ($c = 1$).	Which axis gives robustness?
`random_contrast`	Contrast axis only ($\sigma = 0$).	Same question.
`staircase_f025/f075`	Staircase over 25% / 75% of steps.	Length of the hand-made schedule.

89.1% and `teacher_spg` from 16 to 87.2%, while `sharp` did not change. We therefore recalibrate BatchNorm this way before every evaluation and probe. No test data is used.

The same mechanism acts during training. When a whole batch has contrast c, the first BatchNorm layer largely removes the change of scale, so the network hardly "sees" low contrast. This is an explanation; Chapter 8 ("Which axis and which regime?") gives the experiments that support it.

5. REQUIREMENTS SPECIFICATION

The "system" is the experiment framework. Its one actor is the researcher, who configures a condition (`make_configs`), trains with a schedule or teacher (`train`), resumes from a checkpoint (extends training), evaluates clean and CIFAR-10-C accuracy (`eval`), probes shape bias, filters and PGD (`probe`), and aggregates and plots (`collect`, `digest`). Its one external system is the Colab A100 runtime with Google Drive, used by training, evaluation and probing. Table 5.1 lists the requirements.

Table 5.1. Functional and non-functional requirements.

ID	Requirement	Where / how it is checked
FR1	Degrade a batch with any arm (σ, c) on the GPU; arm $(0, 1)$ is the identity.	`degrade.py`
FR2	Run any of the 20 conditions from one JSON config (condition, seed, budget).	`make_configs.py`, `schedule.py`
FR3	Teacher: Exp3.S with PG, SPG or RPG reward; ALP-GMM as an option.	`teacher.py`, `teacher_alp.py`
FR4	Log the arm of every step and, for teachers, the rewards and policies.	`arms.npy`, `teacher.npz`
FR5	Replay a teacher arm log shuffled in time (`shuffled`).	`schedule.py`
FR6	Evaluate clean accuracy and all 19×5 CIFAR-10-C sets, with seen/unseen means.	`eval.py`
FR7	Probe hybrid shape bias, low-/high-pass accuracy, FGSM and PGD.	`probe.py`
FR8	Aggregate all seeds into mean ± std summaries and per-epoch curves.	`collect.py`, `digest.py`, `blur_curve.py`
NFR1	**Determinism:** same config and seed give the same arm sequence and data order.	Per-epoch seeded generators; condition tests.
NFR2	**Bit-exact resume:** a run stopped and resumed equals an uninterrupted run.	`test_resume.py` for every schedule type.
NFR3	**Fairness:** same steps, same LR schedule and same data order in all conditions.	One training loop; LR is a function of the step only.
NFR4	**Reward measurement is side-effect free:** it does not change BatchNorm statistics or training.	`test_teacher.py`.
NFR5	**No test-data leakage:** rewards use training data; BN recalibration uses clean training images.	Code review; `eval.py`.
NFR6	**Budget and time:** a 30-epoch run in minutes on one A100; stay within 300 compute units.	2.0 s/epoch measured; 6 runs in parallel.
NFR7	**Robust to session loss:** checkpoints every 5 epochs, written atomically to Drive.	F6 was interrupted and resumed.
NFR8	**Testable without a GPU:** CPU smoke run with fake data.	`python -m src.smoke --fake-data`.

6. DESIGN

6.1. Information Structure

The framework has no database. Its information lives in files with a fixed layout (Table 6.1): one JSON config per run, one result folder per run under `results/<budget>/<condition>/<seed>/` (written by `train`), and one summary per budget (written by `collect`).

Table 6.1. Files of the framework.

File	Content
`configs/<short|full>/<condition>_s<seed>.json`	One run: `condition`, `seed`, `epochs`, `norm`, `signal`, `eta`, `importance`, `staircase_frac`, `arms_path`, … A `shuffled` config points (`arms_path`) to the `arms.npy` of the `teacher_pg` run with the same seed.
In `results/<budget>/<condition>/<seed>/`	
`last.pt`	Checkpoint: model, optimiser, grad scaler, schedule state (bandit weights, RNG state, reward buffer), arm log, epoch, config.
`metrics.jsonl`	One row per epoch: `epoch`, `train_loss`, `test_acc`, `sec`.
`arms.npy`	Arm index (0–15) of every training step; arm $= 4\,i_\sigma + i_c$.
`teacher.npz`	Teacher runs only: raw reward per step and the 16-arm policy per step.
`eval.json`	`clean`, accuracy per corruption and severity, `seen_mean`, `unseen_mean`, `all_mean`.
`probe.json`	Low-/high-pass accuracy per σ, hybrid shape bias per σ, FGSM and PGD-10 per ϵ.
`results/<budget>_summary.json`	Mean and std over seeds for every condition and metric.

6.2. Information Flow

Figure 6.1 shows one training step; the schedule is a fixed sequence or a teacher, and fixed schedules ignore the update. After training, the evaluation pipeline runs on the last checkpoint: `last.pt` → recalibrate BN on 100 clean training batches → `eval` (clean + CIFAR-10-C) and `probe` (hybrids, filters, PGD) → `collect` (mean ± std).

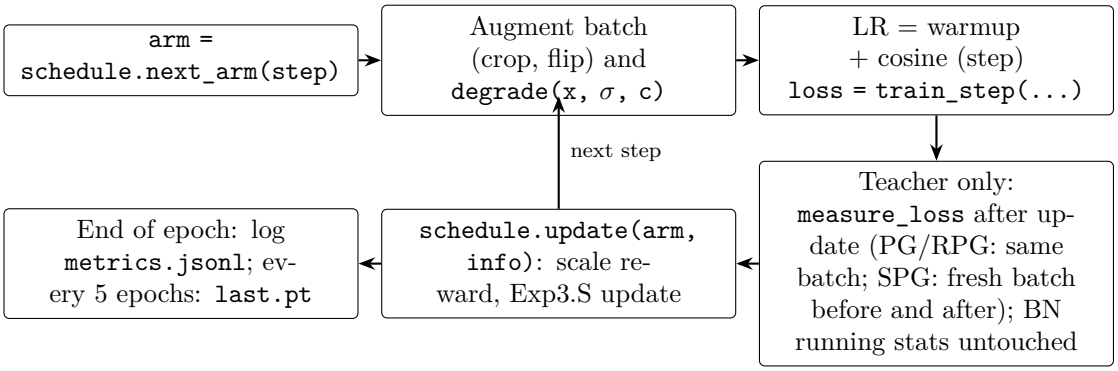


Figure 6.1. One training step. Only teacher schedules use the reward branch.

6.3. System Design

Figure 6.2 shows the modules of `src/` and their dependencies. `train.py` is the centre. It builds the model, gets a schedule from `schedule.make_schedule`, and calls `degrade` on each batch. All schedules share one small interface (`Schedule`): `next_arm(step)`, `update(arm, info)`, `state_dict`/`load_state_dict` for resume, and `save`. Teachers declare which reward measurements they need (`wants_loss_after`, `wants_fresh_batch`); `random_image` declares `per_image`. So the training loop has no condition-specific branches.

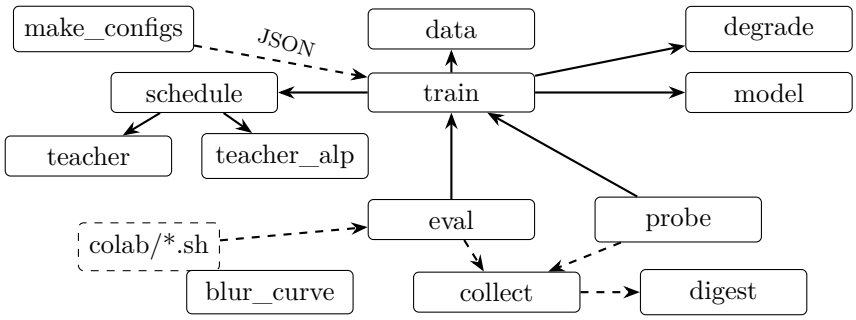


Figure 6.2. Module diagram of `src/`. Solid: Python import. Dashed: data files or script calls. The Colab scripts call `train`, `eval` and `probe`; `blur_curve` reads `arms.npy`.

6.4. User Interface Design (if applicable)

Not applicable: the project has no user interface; the researcher uses command-line modules and Colab cells.

7. IMPLEMENTATION AND TESTING

7.1. Implementation

The code is Python with PyTorch, about 1 200 lines in 14 modules in `src/` and about 340 lines of tests. **GPU-resident data:** the whole CIFAR-10 training set is a `uint8` tensor on the GPU, and augmentation, blur and contrast run there; there is no CPU data loader. **Reproducible randomness:** each epoch uses a generator seeded by (seed, epoch) for data order and augmentation; SPG draws its fresh batch from a second, separate stream, so the training data order matches every other condition. **Side-effect-free reward:** `measure_loss` runs a train-mode forward pass with `track_running_stats` turned off, so measuring the reward does not change BatchNorm statistics; the only GPU synchronisation per step is reading the raw reward. **Resumable runs:** the checkpoint (Table 6.1) is written to a temporary file and renamed (atomic write); metrics rows after the restored epoch are dropped. **Stable bandit:** Exp3.S keeps log-weights normalised to a maximum of 0, so the exponentials do not overflow over long runs. Table 2.2 gives the problems met and their fixes.

7.2. Testing

Unit and integration tests: `pytest` runs 52 test cases (20 test functions, several parametrised over conditions) on the CPU in about 30 s (Table 7.1); the last local run gave `52 passed in 28.59s`. **Smoke test:** `python -m src.smoke --fake-data` runs train, eval and probe end to end on random data on the CPU; on Colab the same command checks the real data. **Acceptance runs:** the first GPU run (`sharp`, 30 epochs, seed 0) reached 93.2% clean accuracy (Section 2.2.2); the 7-condition pilot (seed 0) checked every condition on the GPU and exposed the BatchNorm problem before the main runs.

Table 7.1. Automated tests.

File	What is checked
`test_degrade.py`	Sharp arm is the identity; contrast keeps the channel mean and scales the std by c; blur removes more high-frequency energy as σ grows; arm index mapping; augmentation is a padded crop or its mirror; per-image degradation equals degrading each image with its own arm.
`test_teacher.py`	Reward measurement does not change training; a network that does not learn gives no reward and a uniform policy (PG and SPG); the teacher concentrates on an arm with higher reward and stays near uniform when all arms pay alike; ALP-GMM concentrates on an arm with steady progress.
`test_conditions.py`	Every condition gives a valid arm at every step; staircase goes blurry to sharp and reverse mirrors it; shuffled replays the teacher's arm counts in a new order; axis ablations draw only their axis; `random_image` mixes arms in a batch and is seeded.
`test_probe.py`	Hybrid bias follows the frequency band a model reads; attacks stay in the ϵ-box and hurt only with budget; the probe reports every measure on fake data.
`test_resume.py`	A run stopped and resumed matches an uninterrupted run, for every schedule type.

7.3. Deployment

The code is written and tested (CPU) on a laptop and pushed to the public GitHub repository. Colab Pro is only a launcher: it clones the repository, downloads CIFAR-10 and CIFAR-10-C from Hugging Face, and runs shell scripts in the background (`nohup`) on the A100. Checkpoints and JSON results go to Google Drive (`results/`, `logs/`), so runs survive a closed browser tab and resume after a lost runtime. `docs/report/make_figures.py` makes this report's figures and tables from the result summaries. Commands on Colab (after mounting Drive), then local CPU tests:

```
git clone https://github.com/omarreis01/autofocus-curriculum.git
cd autofocus-curriculum && pip install -q -r requirements.txt
python -m src.data --data-dir /content/data      # ~1.5 min
python -m src.smoke --data-dir /content/data
bash colab/run_budget.sh short 0 1 2             # 7 conditions, 30 epochs
bash colab/run_budget.sh full 0 1 2              # 7 conditions, 100 epochs
bash colab/run_conds.sh short 6 "0 1 2" teacher_rpg teacher_alp ...
python -m src.collect --results $D/results/short --epochs 30 \
    > $D/results/short_summary.json
uv venv --python 3.13 .venv
uv pip install --python .venv/bin/python torch torchvision \
    --index-url https://download.pytorch.org/whl/cpu
uv pip install --python .venv/bin/python -r requirements.txt
.venv/bin/python -m pytest -q tests
```

8. RESULTS

All numbers are accuracies in %, as the mean ± std over 3 seeds; "C" means CIFAR-10-C. Tables A.1 and A.2 in the appendix give every condition and measure, with std.

Main comparison at 100 epochs. Figure 8.1 places the seven main conditions by clean and CIFAR-10-C accuracy. `random` is the best on C: 85.3 seen and 84.4 unseen, with clean accuracy (92.2) only 2.6 points below `sharp`. The teachers (`teacher_pg`, `teacher_spg`) and `shuffled` stay at 85–87 clean, because they spend most of training on the most degraded arm (Figure 8.3). `staircase` ends close to `sharp` (93.1 clean, 77.0 / 78.8 on C): after the blurry phase, the model adapts back to sharp images. `reverse` is good on C-seen (80.9) but the worst on C-unseen (71.9). At 30 epochs the ranking is the same (Table A.1).

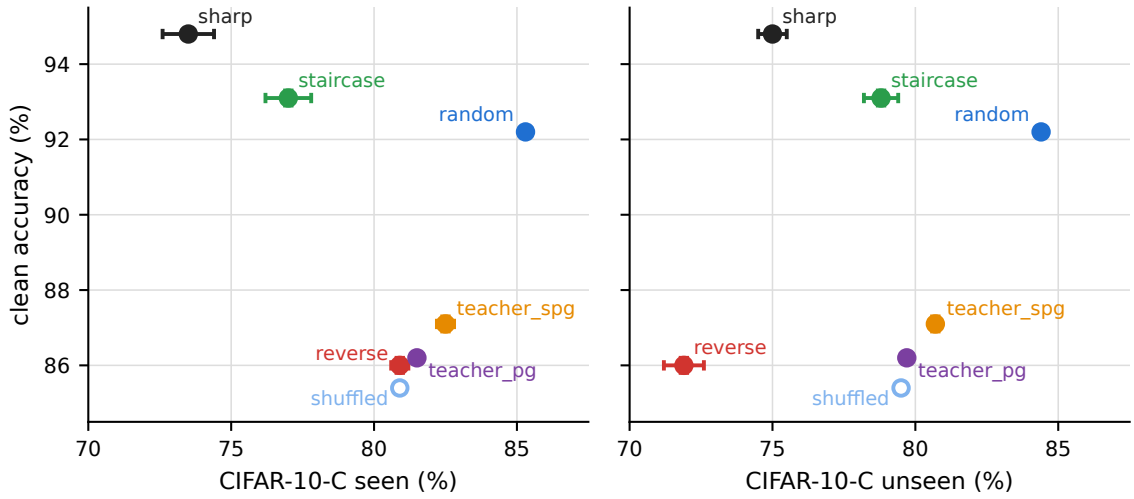


Figure 8.1. Clean vs CIFAR-10-C accuracy at 100 epochs: `random` is best on corruptions and loses little clean accuracy. Bars are ± std (mostly smaller than the dots); axes do not start at zero.

Does the teacher start blurry, and where does it go? This is the direct test of H1. Figure 8.2 shows the mean blur σ and mean contrast of the batches trained in each epoch. If H1 were true, a teacher curve would look like the staircase: high σ

and low c at the start, then sharp. The dotted line is the expected value of uniform choice ($\sigma = 1.31$, $c = 0.54$). None of the teachers starts blurry; all start near uniform. With raw PG and SPG the teacher moves to *more* blur and *less* contrast over time. RPG and ALP-GMM stay near uniform. With $\eta = 0.03$ the teacher reaches the most degraded arm fast and returns towards uniform in the last third; this is not a blurry start either. At 100 epochs the pattern is the same over a longer time: `teacher_pg` stays in the most degraded region from about epoch 10 to 70 and then partly returns.

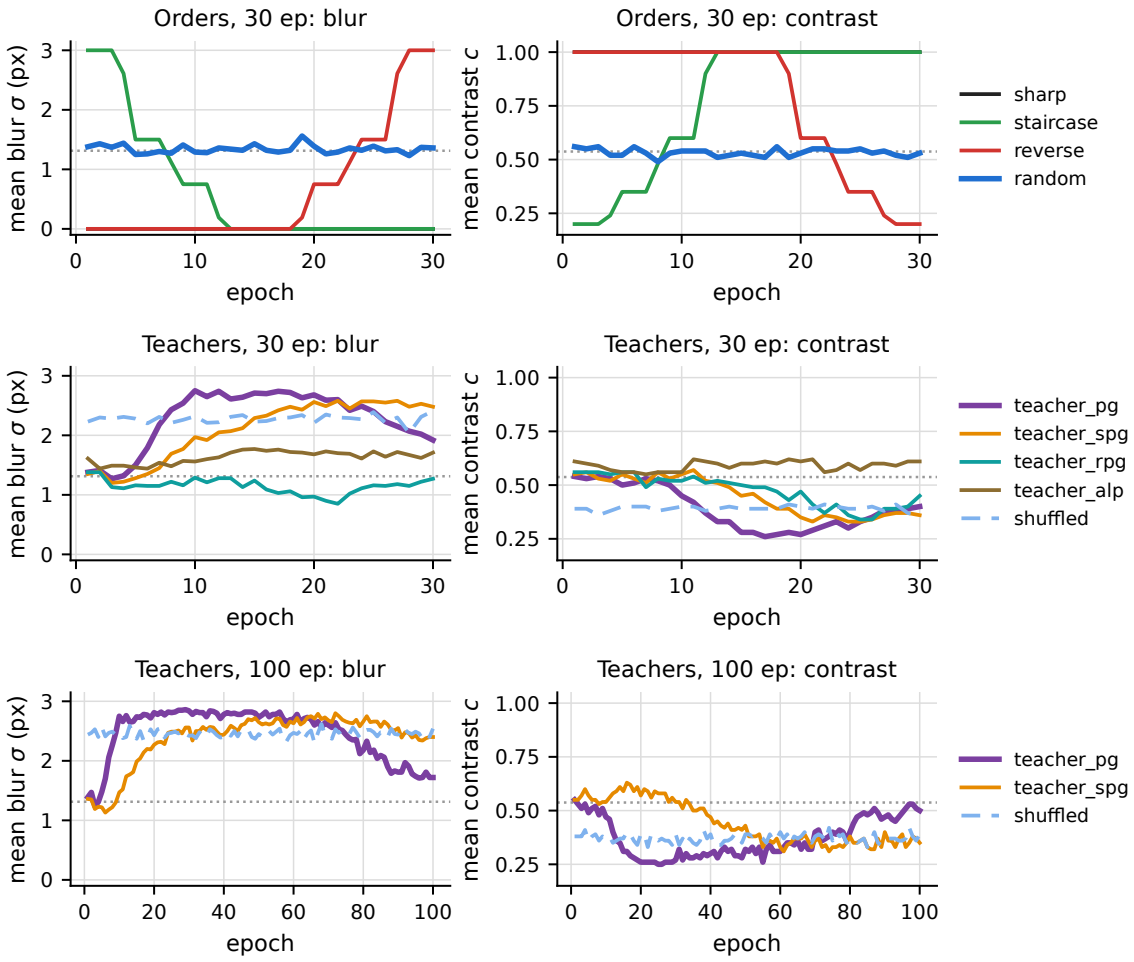


Figure 8.2. Mean blur and contrast per epoch (mean of 3 seeds). Top: the fixed orders at 30 epochs (at 100 epochs they have the same shapes, stretched over 100 epochs); `sharp` is partly hidden under `reverse` (blur) and `staircase` (contrast). Middle and bottom: the teachers at 30 and 100 epochs. The staircase starts blurry by design, but no teacher does; PG and SPG drift towards blur and low contrast, and at 100 epochs `teacher_pg` stays degraded for most of training.

Figure 8.3 shows the share of batches on the most degraded arm ($\sigma = 3, c = 0.2$) in each fifth of training. Raw PG piles up there (71–74% in the middle of a 30-epoch run, up to 83% at 100 epochs); RPG and ALP-GMM stay at the uniform level (1/16). At 30 epochs, the share in the 3rd / 5th fifth is 0.71 / 0.53 for `teacher_pg`, 0.38 / 0.65 for `teacher_spg`, 0.01 / 0.02 for `teacher_rpg`, 0.27 / 0.45 and 0.82 / 0.17 for $\eta = 0.003$ and 0.03, 0.10 / 0.14 without importance weighting (a slow drift), 0.02 / 0.07 for ALP-GMM (near uniform, slightly harder), 0.45 / 0.46 for `shuffled` and 0.06 / 0.08 for `random`. Changing η changes only the speed; both values are still hard-first.

Why the hardest arm? Raw PG is a drop in loss. The loss is highest on the most degraded arm, so one SGD step lowers it most there, and the raw reward is largest. RPG divides by the loss and removes this scale; then the share of the hardest arm falls from 71–74% (PG) to 1–3% (RPG). The RPG experiment supports this explanation. We have not yet analysed the per-arm reward distribution directly (the rewards are stored in `teacher.npz`).

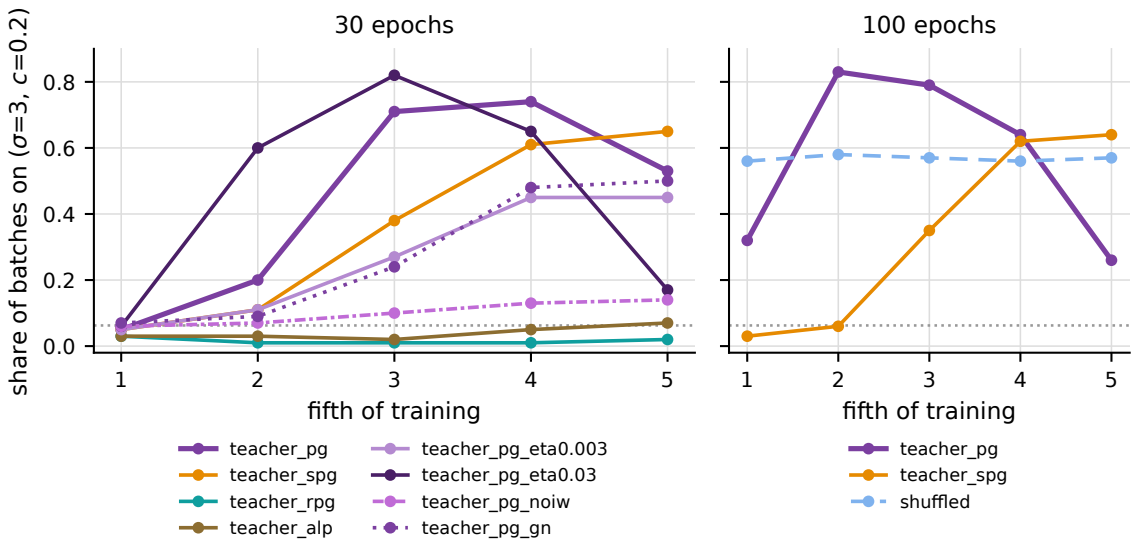


Figure 8.3. Share of batches on the most degraded arm per fifth of training: raw PG concentrates there, while relative PG and ALP-GMM stay at the uniform level (dotted, 1/16).

Three orders: blurry-to-sharp, sharp-to-blurry, random. `staircase` and `reverse` see exactly the same arms, in opposite time order; `random` sees the full grid,

mixed all the time. **Blurry-to-sharp** (`staircase`) keeps clean accuracy (93.1 at 100 epochs). On C it is better than `sharp` (+3.5 / +3.8) and worse than `random` (−8.3 / −5.6). On PGD at 100 epochs it is the highest: 57.8 ± 0.1 vs 56.6 ± 0.5 for `random`. **Sharp-to-blurry** (`reverse`) sees blur in the last epochs, so C-seen is high (80.9). But clean drops (86.0), C-unseen is the worst (71.9), and PGD collapses (18.6, 26 points below `sharp`). **Random** is clearly the best on C, and only 0.9 points below `staircase` on clean. So the direction matters: `reverse` is clearly worse. But blurry-to-sharp is not better than seeing the same kinds of arms mixed.

Which axis and which regime? The 30-epoch ablations isolate the axis and the training regime (Table A.1). **The blur axis gives the robustness:** `random_blur` reaches 82.1 / 81.9 on C, the same as `random`; `random_contrast` reaches 68.2 / 73.6, the same as `sharp`. **Contrast is learned only per image or without BatchNorm:** on the contrast corruption, `random` per batch gets 69.3 and `random_image` gets 88.9; `sharp` with BatchNorm gets 73.8 and with GroupNorm 87.6; batch-level `random_contrast` gets only 76.1. **Costs:** GroupNorm loses about 3 points of clean accuracy at 30 epochs (90.2 vs 93.2), and `random_image` has low PGD accuracy (37.0). With GroupNorm, `teacher_pg` is still hard-first (share of the hardest arm 0.50 in the last fifth). **Staircase length:** a longer blurry phase gives more robustness and less clean accuracy. Even the 75% staircase stays below `random` on C (78.9 / 80.3 vs 81.4 / 82.2). This fits an effect of how much blur is seen.

Shape bias and adversarial robustness. Shape bias grows with the time spent on the most degraded arm (Figure 8.4): `sharp` 6.2, `random` 22.7, `teacher_pg` 54.2, `shuffled` 59.7 (30 epochs). `shuffled` sees the same arms as `teacher_pg` without order, and its shape bias is slightly higher. The cost is accuracy on high frequencies: high-pass $\sigma = 1$ accuracy is 80.2 for `sharp`, 39.7 for `teacher_pg` and 35.7 for `shuffled`. Blur helps PGD: `sharp` 34.8, `random` 59.5 (30 epochs). `reverse` is the worst (26.7 / 18.6 at 30 / 100 epochs).

How sure are we? Table 8.1 grades each claim by its evidence.

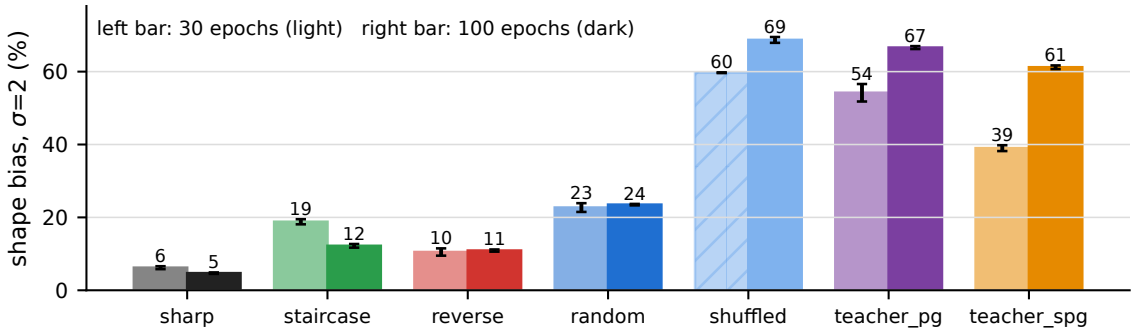


Figure 8.4. Shape bias ($\sigma = 2$ hybrids) for the main conditions: it follows exposure to heavy blur, not order; `shuffled` (hatched) is at least as high as `teacher_pg`.

Table 8.1. Strength of the main claims.

Strength	**Claim and evidence**
Solid	**No teacher started blurry.** Seven teacher variants (PG, SPG, RPG, ALP-GMM, two η values, no importance weighting) plus PG with GroupNorm, each with 3 seeds. All start near uniform. H1 is rejected in this set-up.
Solid	**At 100 epochs no order beat `random` on CIFAR-10-C.** `random` 85.3 / 84.4; the closest teacher, `teacher_spg`, 82.5 / 80.7. The gap is far above the std ($\leq$0.9).
Solid	**Blur gives the robustness, contrast does not.** `random_blur` $\approx$ `random`; `random_contrast` $\approx$ `sharp` (30 epochs).
Solid	**Contrast is learned only per image or with GroupNorm** (88.9 and 87.6 vs 69.3 per batch on the contrast corruption). The BatchNorm mechanism is an explanation; we did not analyse activations.
Fine distinction	**`teacher_pg` and `shuffled` are close, not equal.** On robustness the gap is small and favours the teacher: clean +0.6 / +0.8, C-seen +0.4 / +0.6, C-unseen +0.3 / +0.2 (30 / 100 epochs), with std 0.1–0.3, which is weak evidence with 3 seeds. On shape bias and PGD `shuffled` is ahead: shape bias 59.7 vs 54.2 (30 ep) and 68.7 vs 66.6 (100 ep); PGD 51.2 vs 49.3 (30 ep) and 49.3 vs 48.5 (100 ep). The teacher's order adds no measurable robustness, but we cannot say that order never matters.
Fine distinction	**Direction matters.** `reverse` is clearly the worst order (C-unseen 71.9, PGD 18.6 at 100 epochs). So the claim is "blurry-to-sharp is not better than mixing", not "order is irrelevant".
Fine distinction	**"Random is best" holds among the main seven at 100 epochs on CIFAR-10-C.** At 30 epochs some new variants tie or win in part: `random_image` 86.9 C-seen (vs 81.4) but 81.2 C-unseen (vs 82.2); `teacher_rpg` 90.0 / 81.1 / 82.4 $\approx$ `random`; `random_blur` +1.4 clean, +0.7 C-seen. These were not run at 100 epochs. On PGD at 100 epochs `staircase` is 1.2 points above `random`.
Explanation	**The teacher goes to the hardest arm because the loss drop is largest there.** Supported by RPG (hard-first disappears when the scale is removed), not by a direct per-arm reward analysis.

9. CONCLUSION

H1 is rejected in our set-up: no teacher variant started blurry; raw prediction gain sends the teacher to the most degraded arm, where the loss drop is largest, while relative progress and ALP-GMM stay near uniform. **H2: exposure, not order.** Corruption robustness, PGD robustness and the shape-bias proxy grow with exposure to blur; the teacher's order adds no measurable robustness over the same arms shuffled; a uniform random arm per batch was the best of the seven main conditions on CIFAR-10-C at 100 epochs, though direction matters (Table 8.1). These results agree with earlier warnings that curriculum gains often come from exposure [10,11] and that mixed blur is a strong baseline [15].

Limitations. (i) One dataset (CIFAR-10) and one model (ResNet-18); at 32 pixels, $\sigma = 3$ blur is very strong. (ii) Three seeds; differences below 1 point are weak evidence. (iii) The shape-bias measure is a frequency proxy, not Geirhos' test. (iv) BatchNorm recalibration is a choice; another fix (for example mixed batches during training) may give different clean numbers. (v) ALP-GMM samples a continuous space but training rounds to the 16 arms; a continuous version was not tried. (vi) The `random_image` arm log stores only the first image's arm (an unbiased but incomplete histogram). (vii) We did not run the noise tests of Arslan et al. [3]; the CIFAR-10-C noise corruptions are only a close match.

Future work. (i) Run `random_image`, `random_blur`, `teacher_rpg` and `teacher_alp` at 100 epochs (about 40 minutes on an A100) to test "random is best" against them. (ii) A per-image teacher (a distribution over arms for each batch), to combine the contrast benefit of per-image mixing with a teacher. (iii) A transfer-progress reward: the loss drop on a clean (sharp) validation batch. It steers the teacher to the arm that helps sharp vision most, not where loss falls most; robustness still stays out of the reward. (iv) A per-arm reward analysis from `teacher.npz`, to test the hard-first explanation directly. (v) A larger dataset (for example Tiny-ImageNet), if the budget allows.

REFERENCES

1. Vogelsang, L., S. Gilad-Gutnick, E. Ehrenberg, A. Yonas, S. Diamond, R. Held and P. Sinha, "Potential Downside of High Initial Visual Acuity", *Proceedings of the National Academy of Sciences*, Vol. 115, No. 44, pp. 11333–11338, 2018.

2. Jang, H. and F. Tong, "Convolutional Neural Networks Trained with a Developmental Sequence of Blurry to Clear Images Reveal Core Differences between Face and Object Processing", *Journal of Vision*, Vol. 21, No. 12, 2021, Article 6, doi:10.1167/jov.21.12.6.

3. Arslan, S. S., H. Jang and P. Sinha, "Internal Neural Noise Progression for Emergent Classification Robustness", *Conference on Cognitive Computational Neuroscience (CCN)*, 2024.

4. Graves, A., M. G. Bellemare, J. Menick, R. Munos and K. Kavukcuoglu, "Automated Curriculum Learning for Neural Networks", *Proceedings of the 34th International Conference on Machine Learning (ICML)*, PMLR, 2017, arXiv:1704.03003.

5. Krizhevsky, A., *Learning Multiple Layers of Features from Tiny Images*, Technical Report, University of Toronto, 2009.

6. Hendrycks, D. and T. Dietterich, "Benchmarking Neural Network Robustness to Common Corruptions and Perturbations", *International Conference on Learning Representations (ICLR)*, 2019.

7. He, K., X. Zhang, S. Ren and J. Sun, "Deep Residual Learning for Image Recognition", *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778, IEEE, 2016.

8. Auer, P., N. Cesa-Bianchi, Y. Freund and R. E. Schapire, "The Nonstochastic Multiarmed Bandit Problem", *SIAM Journal on Computing*, Vol. 32, No. 1, pp. 48–77, 2002.

9. Portelas, R., C. Colas, K. Hofmann and P.-Y. Oudeyer, "Teacher Algorithms for Curriculum Learning of Deep RL in Continuously Parameterized Environments", *Proceedings of the Conference on Robot Learning (CoRL)*, PMLR, 2019, arXiv:1910.07224.

10. Wu, X., E. Dyer and B. Neyshabur, "When Do Curricula Work?", *International Conference on Learning Representations (ICLR)*, 2021, arXiv:2012.03107.

11. Katzhendler, G. and D. Weinshall, "Potential Upside of High Initial Visual Acuity?", *Proceedings of the National Academy of Sciences*, Vol. 116, No. 38, pp. 18765–18766, 2019.

12. Wang, Y., Y. Yue, R. Lu, T. Liu, Z. Zhong, S. Song and G. Huang, "EfficientTrain: Exploring Generalized Curriculum Learning for Training Visual Backbones", *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, IEEE, 2023, arXiv:2211.09703.

13. Wang, Y., Y. Yue, R. Lu, Y. Han, S. Song and G. Huang, "EfficientTrain++: Generalized Curriculum Learning for Efficient Visual Backbone Training", *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2024, arXiv:2405.08768.

14. Hacohen, G. and D. Weinshall, "On The Power of Curriculum Learning in Training Deep Networks", *Proceedings of the 36th International Conference on Machine Learning (ICML)*, PMLR, 2019, arXiv:1904.03626.

15. Jang, H. and F. Tong, "Improved Modeling of Human Vision by Incorporating Robustness to Blur in Convolutional Neural Networks", *Nature Communications*, Vol. 15, 2024, Article 1989, doi:10.1038/s41467-024-45679-0.

16. Geirhos, R., P. Rubisch, C. Michaelis, M. Bethge, F. A. Wichmann and W. Brendel, "ImageNet-Trained CNNs are Biased towards Texture; Increasing Shape Bias Improves Accuracy and Robustness", *International Conference on Learning Representations (ICLR)*, 2019.

17. Wang, H., X. Wu, Z. Huang and E. P. Xing, "High-Frequency Component Helps Explain the Generalization of Convolutional Neural Networks", *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, IEEE, 2020.

18. Madry, A., A. Makelov, L. Schmidt, D. Tsipras and A. Vladu, "Towards Deep Learning Models Resistant to Adversarial Attacks", *International Conference on Learning Representations (ICLR)*, 2018.

19. Ioffe, S. and C. Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift", *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, pp. 448–456, PMLR, 2015.

20. Wu, Y. and K. He, "Group Normalization", *Proceedings of the European Conference on Computer Vision (ECCV)*, pp. 3–19, Springer, 2018.

APPENDIX A: FULL RESULT TABLES AND EXPERIMENT LOG

Mean ± std over 3 seeds, in %. Noise: mean of the 4 noise corruptions. C-contr.: contrast corruption. Shape: hybrid shape bias at $\sigma = 2$. HP $\sigma 1$: high-pass accuracy at $\sigma = 1$. PGD: PGD-10 at $\epsilon = 1/255$ on 2 000 test images. Source: the result summaries (`results/{short,full}_summary.json`), as copied into `docs/reports/pilot-short-s0.html`. Table A.3 summarises the experiment log; the full log is in `docs/experiments.md`.

Table A.1. All 20 conditions at 30 epochs.

Condition	Clean	C-seen	C-unseen	Noise	C-contr.	Shape	HP $\sigma 1$	PGD
`sharp`	93.2 ±0.1	69.6 ±0.4	73.3 ±0.1	55.5 ±0.1	73.8 ±0.6	6.2 ±0.4	80.2 ±0.8	34.8 ±0.6
`staircase`	90.7 ±0.2	75.8 ±0.7	78.7 ±0.0	68.3 ±0.4	68.8 ±0.4	18.8 ±0.7	65.7 ±1.2	55.0 ±0.5
`reverse`	84.4 ±0.2	78.1 ±0.1	71.3 ±0.8	55.0 ±2.4	74.1 ±0.4	10.5 ±1.0	75.0 ±0.5	26.7 ±1.2
`random`	89.4 ±0.2	81.4 ±0.3	82.2 ±0.1	76.4 ±0.1	69.3 ±0.8	22.7 ±1.2	66.1 ±2.4	59.5 ±1.0
`shuffled`	84.2 ±0.3	79.7 ±0.2	78.5 ±0.1	75.4 ±0.2	63.7 ±0.5	59.7 ±0.1	35.7 ±0.9	51.2 ±0.1
`teacher_pg`	84.8 ±0.3	80.1 ±0.2	78.8 ±0.2	74.8 ±0.3	64.1 ±0.3	54.2 ±2.4	39.7 ±1.5	49.3 ±0.9
`teacher_spg`	87.0 ±0.1	81.0 ±0.2	80.3 ±0.1	75.4 ±0.3	66.3 ±0.9	39.0 ±0.8	52.9 ±1.0	54.0 ±0.1
`teacher_rpg`	90.0 ±0.2	81.1 ±0.4	82.4 ±0.1	76.3 ±0.6	70.5 ±0.6	17.6 ±1.9	66.4 ±1.9	58.0 ±0.9
`teacher_pg_eta0.003`	86.6 ±0.0	81.1 ±0.1	80.2 ±0.2	75.5 ±0.5	66.1 ±0.5	44.2 ±1.0	49.2 ±1.5	53.3 ±1.3
`teacher_pg_eta0.03`	84.6 ±0.3	79.9 ±0.2	78.8 ±0.2	75.5 ±0.3	64.1 ±0.7	55.7 ±1.8	38.4 ±3.6	51.3 ±0.4
`teacher_pg_noiw`	88.8 ±0.1	81.6 ±0.2	81.7 ±0.0	75.9 ±0.1	68.8 ±0.4	28.4 ±0.2	61.6 ±1.5	59.7 ±0.8
`teacher_alp`	89.0 ±0.5	82.1 ±0.1	82.0 ±0.4	76.4 ±0.8	67.7 ±0.8	29.9 ±0.5	61.3 ±1.5	56.6 ±1.1
`random_image`	89.6 ±0.0	86.9 ±0.1	81.2 ±0.2	71.7 ±0.7	88.9 ±0.1	22.3 ±0.8	81.8 ±0.1	37.0 ±1.0
`random_blur`	90.8 ±0.3	82.1 ±0.3	81.9 ±0.2	73.8 ±0.1	67.2 ±0.6	23.3 ±1.2	66.8 ±0.9	56.4 ±0.5
`random_contrast`	92.7 ±0.2	68.2 ±0.3	73.6 ±0.3	57.4 ±1.1	76.1 ±1.0	4.2 ±0.1	81.6 ±1.5	31.2 ±1.2
`sharp_gn`	90.2 ±0.3	79.6 ±0.4	78.8 ±0.5	69.4 ±1.4	87.6 ±0.6	14.6 ±1.4	72.9 ±0.9	38.2 ±1.5
`random_gn`	84.7 ±0.2	83.0 ±0.2	79.7 ±0.1	76.2 ±0.1	84.8 ±0.2	41.1 ±1.6	70.7 ±1.2	49.4 ±0.3
`teacher_pg_gn`	79.0 ±0.3	78.3 ±0.3	74.6 ±0.3	72.3 ±0.4	79.4 ±0.3	59.3 ±0.8	60.7 ±1.2	43.2 ±0.6
`staircase_f025`	92.0 ±0.1	71.0 ±0.9	74.8 ±0.4	58.9 ±1.1	71.9 ±0.1	9.6 ±0.3	76.9 ±0.6	41.0 ±0.8
`staircase_f075`	88.5 ±0.1	78.9 ±0.1	80.3 ±0.2	75.0 ±0.2	64.5 ±0.8	33.2 ±0.8	50.8 ±3.8	56.1 ±0.7

Table A.2. The seven main conditions at 100 epochs.

Condition	Clean	C-seen	C-unseen	Noise	C-contr.	Shape	HP σ1	PGD
`sharp`	94.8 ±0.0	73.5 ±0.9	75.0 ±0.5	54.3 ±1.3	76.1 ±0.3	4.7 ±0.2	85.0 ±0.2	44.9 ±0.2
`staircase`	93.1 ±0.2	77.0 ±0.8	78.8 ±0.6	64.0 ±1.6	71.5 ±0.2	12.2 ±0.5	75.7 ±0.4	57.8 ±0.1
`reverse`	86.0 ±0.2	80.9 ±0.3	71.9 ±0.7	52.1 ±2.1	74.4 ±0.4	10.9 ±0.3	74.9 ±1.5	18.6 ±2.0
`random`	92.2 ±0.0	85.3 ±0.2	84.4 ±0.1	76.7 ±0.2	72.9 ±0.7	23.5 ±0.2	65.9 ±1.1	56.6 ±0.5
`shuffled`	85.4 ±0.1	80.9 ±0.1	79.5 ±0.1	76.7 ±0.4	63.1 ±0.9	68.7 ±0.8	27.6 ±4.8	49.3 ±1.2
`teacher_pg`	86.2 ±0.1	81.5 ±0.2	79.7 ±0.1	76.1 ±0.6	63.6 ±0.4	66.6 ±0.4	26.9 ±0.1	48.5 ±0.6
`teacher_spg`	87.1 ±0.2	82.5 ±0.3	80.7 ±0.1	76.7 ±0.3	65.8 ±0.8	61.2 ±0.5	34.4 ±1.2	46.9 ±1.0

Table A.3. Experiment log (2026).

#	Date	Trial	Result
1	11-03	Sharp, 30 ep, seed 0	Clean 93.2% (target ∼93).
2	11-03	Eager vs `torch.compile`	2.0 vs 1.7 s/epoch; compile costs 30 s; eager chosen.
3	11-03	Pilot: 7 conditions, seed 0	Mixed conditions 13–60% clean; BatchNorm suspected.
4	11-03	BN recalibration	Confirmed: `random` 60→89, `teacher_spg` 16→87 clean.
5	11-04	F5: 7 × 3 × 30 ep	`random` best on C; teacher goes to the hardest arm.
6	11-08	F6: 7 × 3 × 100 ep	Same ranking as F5.
7	11-08	`teacher_rpg`	Hardest-arm share 1–3% (PG: 71–74%); ≈ `random`.
8	11-08	η 0.003 / 0.03	Only speed changes; both still hard-first.
9	11-08	No importance weighting	Slow drift (14% late).
10	11-08	ALP-GMM	Near uniform, slightly harder; ≈ `random`.
11	11-08	Random arm per image	Contrast corruption 88.9 (per batch 69.3); PGD low (37.0).
12	11-08	GroupNorm	`sharp_gn` contrast 87.6 (BN 73.8); clean −3; teacher still hard-first.
13	11-08	Blur-only / contrast-only	Blur ≈ `random`; contrast-only ≈ `sharp`.
14	11-08	Staircase 25% / 75%	Longer blurry phase: more robust, less clean.
15	11-08	Shape bias, low/high-pass	Follows blur exposure, not order.
16	11-08	FGSM, PGD-10	Blur helps PGD; FGSM likely gradient-masked, not used.
17	11-08	OOM at 12 parallel runs	3 runs re-run at 6 parallel from checkpoint.